

ECON-520

Ciencia de Datos para Economía y Negocios Clases 03-04

Viktoriya Semeshenko
Sergio De Raco





```
Archivo Edición Pestañas Ayuda
[sonlink@localhost ~]$ ./telegram
Telegram-client version 0.8.0-beta, Copyright (C) 2013 Vitaly Volynov.
Telegram-client comes with ABSOLUTELY NO WARRANTY; for details type 'show_license'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'show_license' for details.
/home/sonlink/.telegram/ created
/home/sonlink/.telegram/downloads/ created
Telephone number (with '+' sign):
*** phone registered
*** sending code
*** send_code: dc_num = 4
code from sms:
User Alfonso aka Son Link: 0 unread
User Monica/1: 1 unread
[10:55] User Alfonso aka Son Link is now offline
```

Clase 01/02 (cont.: git + T01) Workplace. Instalación y setup.

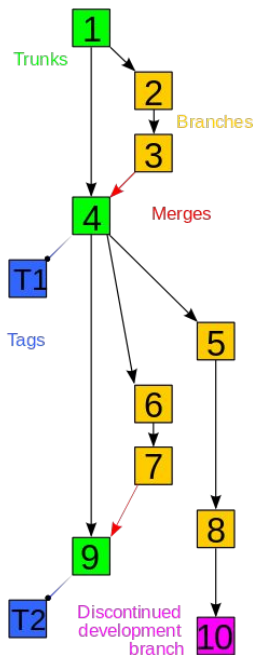


Git: control de versiones



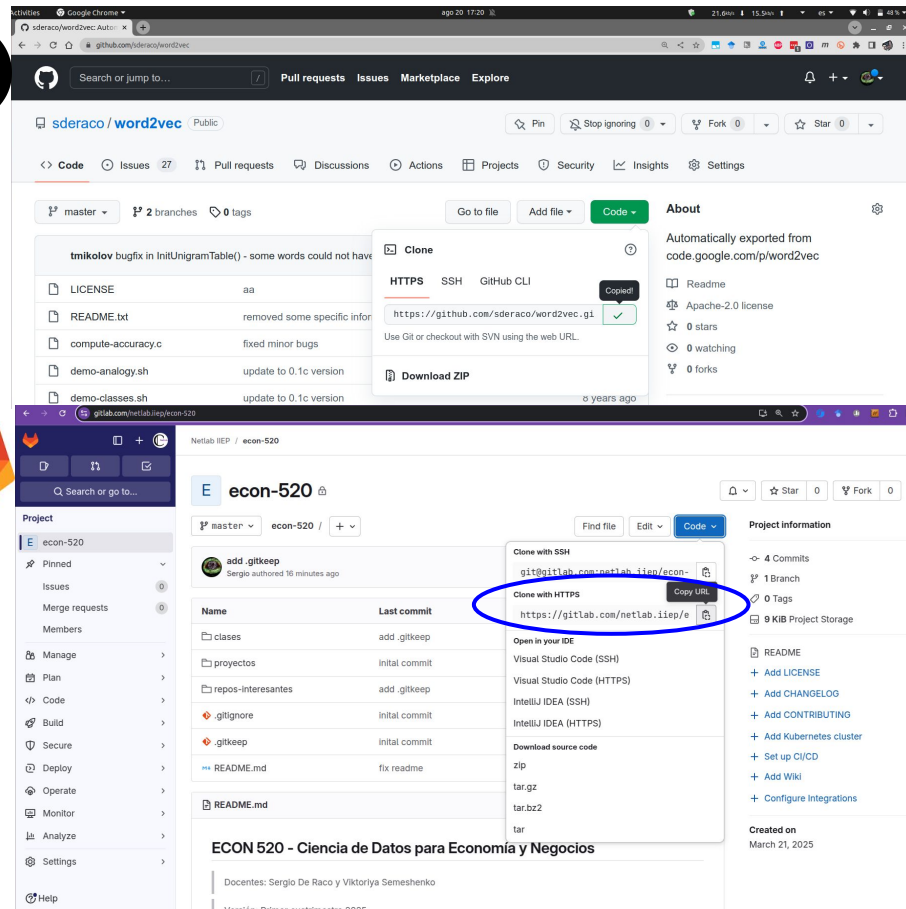
Un sistema para gestionar control de cambios en colecciones de información:

- Workflows
- Versiones
- Colaboración

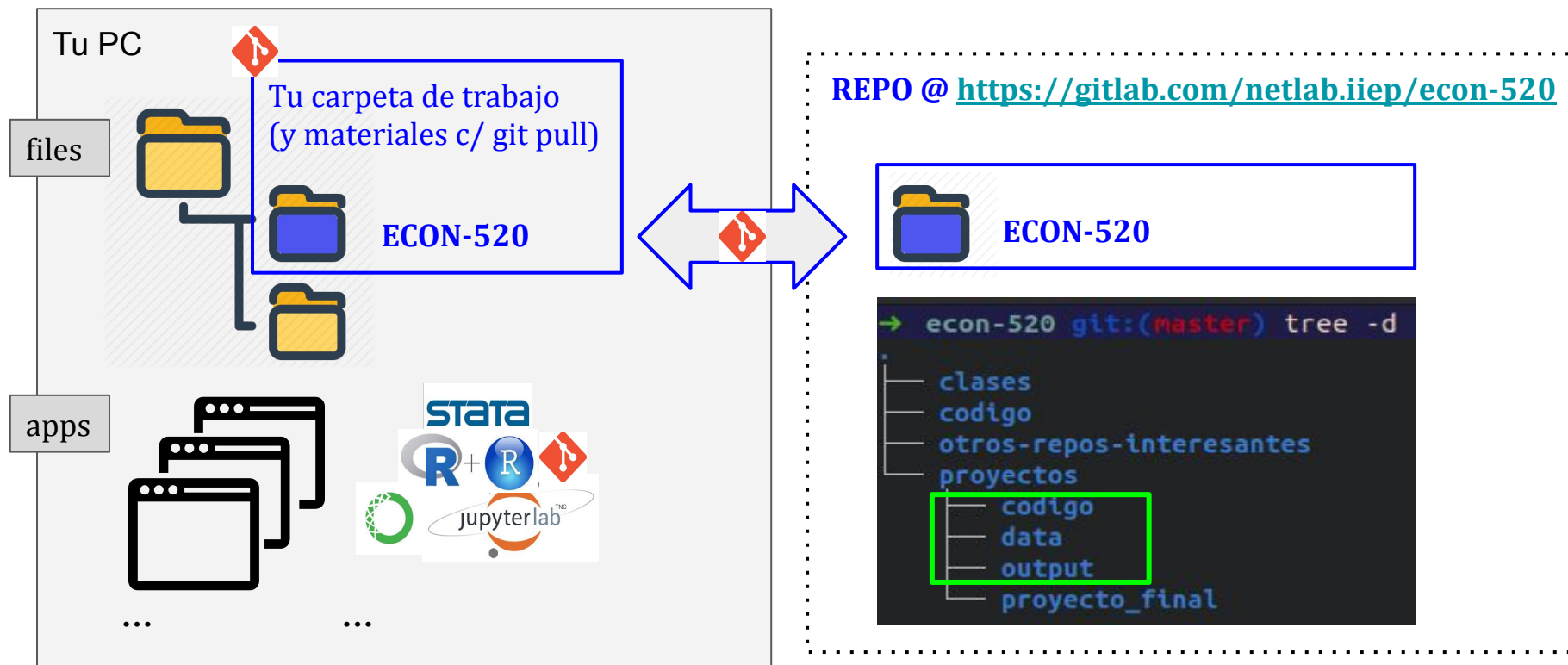


Referencias/Ayuda

- [Gitlab tutorials](#)
- [Simple guide](#)
- [Turing way \(VCS\)](#)



Entorno de trabajo: git (clone) repo del curso



Mini-guía para **git** que usaremos aquí

PC usuario `C:\Users\YO\..\[X]` [tu carpeta creada para el curso]

- 1 - abrir explorador de archivos ( +E)
- 2 - entrar a [X]
- 3 - abrir ventana de terminal en [X] (Shift+ )

PRIMER CLONE

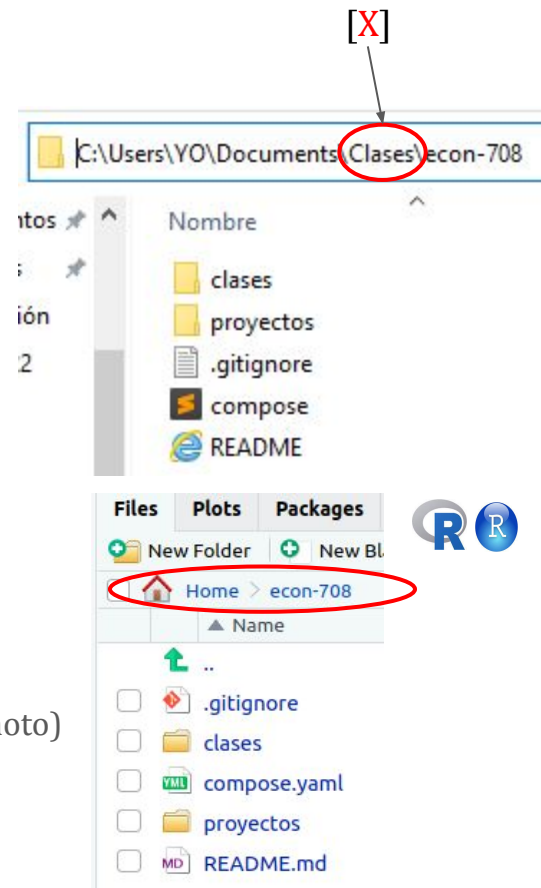
- 3.1 `git clone https://gitlab.com/netlab.iiep/econ-520.git`
- 3.2 `cd econ-520`

CADA PULL

- 3.4 `git pull`
 - Sin error: tenes carpeta actualizada
 - Error (Aborting! Please commit changes or stash)
 - `git status` (leer salida: archivos con cambios respecto del repo remoto)
 - `git add .` (si hay salidas en rojo/verde: OJO el "." va)
 - `git stash` (guarda los cambios en una version local que no sincroniza con remoto)
 - `git pull`



MUY UTIL: [git cheat sheet](#)



Clonar repo <https://gitlab.com/netlab.iiep/econ-520> ??

The screenshot shows the GitLab web interface for the repository 'econ-520' under the 'Netlab IIEP' namespace. The browser's address bar shows the URL 'gitlab.com/netlab.iiep/econ-520'. The repository page includes a sidebar with navigation links, a main content area with a file list, and a right sidebar with project information. The 'Code' dropdown menu is open, displaying options for cloning the repository via SSH or HTTPS, opening in an IDE, and downloading the source code. The 'Clone with HTTPS' option is circled in blue, and the 'Copy URL' button next to it is highlighted.

Project: **econ-520**

add .gitkeep
Sergio authored 16 minutes ago

Name	Last commit
clases	add .gitkeep
proyectos	inital commit
repos-interesantes	add .gitkeep
.gitignore	inital commit
.gitkeep	inital commit
README.md	fix readme

README.md

ECON 520 - Ciencia de Datos para Economía y Negocios

Docentes: Sergio De Raco y Viktoriya Semeshenko

Project information

- 4 Commits
- 1 Branch
- 0 Tags
- 9 KIB Project Storage

README

- + Add LICENSE
- + Add CHANGELOG
- + Add CONTRIBUTING
- + Add Kubernetes cluster
- + Set up CI/CD
- + Add Wiki
- + Configure Integrations

Created on
March 21, 2025

Puesta a punto con Github!

- [Crear cuenta](#) de usuario con correo de la facultad
 - (@[campus.economicas.uba.ar](#), después pueden agregar otro correo alternativo)
- [Crear un repo](#) público (“repo-curso-2026”) con el usuario creado
- Copiar el nombre del repo a la [planilla de repos](#)
 - la usaremos en el curso para monitorear el cumplimiento de las tareas
 - [p.ej.](#) copiar `https://github.com/tu-usuario/repo-curso-2026`
- EXTRA: tramitar la [licencia de estudiantes](#) Copilot en Github Education

Instalación y puesta a punto del entorno de trabajo están terminadas

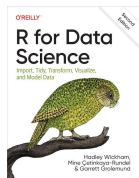
¡ESTAMOS LISTOS!

Clonar repos

>_

Abrir terminal

- `cd econ-520/repos-interesantes`
- `git clone https://github.com/hadley/r4ds.git`



Libro [R for Data Science \(2e\)](#)

- `git clone [cualquier otro repo que encuentren que les guste!]`



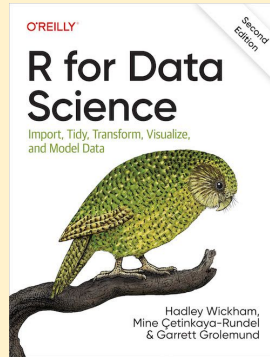
Tareas

- **Tarea 00 - Init**

- Crear repo personal (repo-curso-2026) en github.com (en clase)
- Crear una carpeta para “Tareas” que usarán para guardas los archivos de las tareas asignadas y luego harán “git push” para entregarlas.

- **Tarea 01 - Práctica de CLI**

- Hacer las partes prácticas de este [Tutorial Linux](#) desde la sección “Abrir la terminal” hasta la sección de “Un poco de plomería” y grabar el [log de comandos y salidas](#) en un archivo ([p.ej.](#) “tarea01.txt” en la carpeta Tareas creada antes o en una subcarpeta Tareas/Tarea01



<https://bit.ly/econ520-2025c2-clases03>

Clase 03

Ejemplo tidyverse. Introducción a la programación en R.

Aprender R

[CRAN](https://cran.r-project.org/)

[Comprehensive R Archive Network](https://comprehenstive.r-project.org/)



Dos enfoques de aprendizaje

- R Base
- Tidyverse

Característica	Base R	Tidyverse
Consistencia	La sintaxis puede ser inconsistente entre funciones.	Sintaxis y filosofía de diseño coherentes en todo el ecosistema.
Legibilidad	El código anidado puede ser difícil de leer.	El uso del "pipe" (<code>%>%</code> o <code> ></code>)
Dependencias	Mínimas, solo la instalación base de R.	Requiere la instalación y carga de paquetes adicionales.
Velocidad	Generalmente muy rápido, especialmente para operaciones vectorizadas.	Puede ser ligeramente más lento en algunos casos debido a la sobrecarga, pero a menudo es imperceptible.
Curva de Aprendizaje	Puede ser más empinada al principio debido a su sintaxis variada y a veces críptica.	Considerado más intuitivo y fácil de aprender para los principiantes en ciencia de datos.

Introducción a la programación en R (base)



[R Tutorial](#) (W3 Schools)

- types: [data types](#), [numbers](#), [strings](#), [booleans](#), [operators](#)
- conditionals + flow control: [if else](#), [while loop](#), [for loop](#)
- data structures: [vectors](#), [lists](#), [matrices](#), [arrays](#), [data frames](#), [factor s](#)

para tarea 02!



Otros recursos

- [An introduction to R](#) (oficial, CRAN)
- [R para Principiantes](#) (contrib, CRAN)

Introducción a los Tipos de Datos

- Concepto de variable en R y asignación básica.

```
# Asignación de un valor a una variable  
encuestado_id <- 1045
```

- Tipos básicos: numeric, integer, character (strings), logical.

```
ingreso <- 350000.50 # numeric  
miembros_hogar <- 4L # integer (la 'L' fuerza que sea entero)  
estado <- "Ocupado" # character  
busca_trabajo <- FALSE # logical
```

- Función clave: class() para identificar el tipo.

```
class(ingreso) # Devuelve: "numeric"  
class(busca_trabajo) # Devuelve: "logical"
```

Números y Cadenas de Texto

- Diferencia entre numeric e integer en la práctica.

```
horas_trabajadas <- 40.5 # Puede tener decimales
```

```
edad_anios <- 28L      # Edad exacta en años completos
```

- Manejo de Strings (cadenas de texto).

```
sector_actividad <- "Comercio"
```

```
categoria_ocupacional <- 'Cuentapropista'
```

- Funciones útiles: nchar(), grepl(), paste().

```
# Unir textos para un reporte
```

```
paste("Sector:", sector_actividad, "-", categoria_ocupacional)
```

```
# Buscar un patrón (¿Contiene la palabra "propia"?)
```

```
grepl("propia", "Cuenta propia con local") # Devuelve: TRUE
```

Booleanos y Operadores

- Valores lógicos y operadores aritméticos.

```
salario_mensual <- 450000
```

```
salario_anual <- salario_mensual * 13 # Incluye aguinaldo
```

- Operadores de comparación (==, >, <).

```
es_mayor_edad <- edad_anios >= 18
```

```
es_desocupado <- estado == "Desocupado"
```

- Operadores lógicos (& AND, | OR, ! NOT).

```
# Verificamos si pertenece a la Población Económicamente Activa (PEA)
```

```
es_pea <- (estado == "Ocupado" | estado == "Desocupado") & edad_anios >= 16
```


Toma de decisiones (If...Else)

- Estructura básica de if.

```
if (salario_mensual < 200000) {  
  print("Por debajo del salario mínimo")  
}
```

- Añadiendo else if y else para clasificar un ingreso.

```
if (salario_mensual > 800000) {  
  decil <- "Alto"  
} else if (salario_mensual >= 300000) {  
  decil <- "Medio"  
} else {  
  decil <- "Bajo"  
}
```

Bucles - While Loop

- Iteración mientras se cumpla una condición (Simulación de búsqueda).

```
meses_busqueda <- 0
while (meses_busqueda < 3) {
  print(paste("Mes", meses_busqueda, ": Buscando empleo..."))
  meses_busqueda <- meses_busqueda + 1
}
```

- Uso de break para salir del bucle prematuramente.

```
# Simulación: encuentra trabajo al mes 2
meses_busqueda <- 0
while (TRUE) {
  meses_busqueda <- meses_busqueda + 1
  if (meses_busqueda == 2) {
    print("¡Empleo encontrado!")
    break
  }
}
```

Bucles - For Loop

- Iteración sobre un vector de datos.

```
# Vector con salarios horarios
```

```
salarios_hora <- c(1500, 2200, 1800, 3100)
```

```
for (salario in salarios_hora) {
```

```
  print(salario * 8) # Imprime el salario por jornada de 8 horas
```

```
}
```

- Iteración usando índices (muy común en limpieza de datos).

```
for (i in 1:length(salarios_hora)) {
```

```
  # Aplicar un aumento del 10% a cada elemento
```

```
  salarios_hora[i] <- salarios_hora[i] * 1.10
```

```
}
```

Vectores y Listas

- Vectores: Elementos del mismo tipo (ideal para columnas de variables).

```
edades_hogar <- c(45, 42, 16, 12)
```

```
promedio_edad <- mean(edades_hogar)
```

- Listas: Contenedores heterogéneos (ideal para perfiles completos).

```
jefe_hogar <- list(
```

```
  id = 101,
```

```
  nombre = "Carlos",
```

```
  edades_familia = edades_hogar,
```

```
  es_propietario = TRUE
```

```
)
```

Matrices y Arrays

- Matrices: Estructura 2D del mismo tipo (ej. matriz de transición de ocupaciones).

```
# Transición Ocupado/Desocupado (T1 a T2)
```

```
datos_transicion <- c(80, 20, 15, 85)
```

```
matriz_transicion <- matrix(datos_transicion, nrow = 2, byrow = TRUE)
```

- Arrays: Estructuras N-dimensionales (ej. datos de panel por varios trimestres).

```
# 2 filas, 2 columnas, 3 "capas" (trimestres)
```

```
panel_laboral <- array(1:12, dim = c(2, 2, 3))
```

Data Frames

- La estructura central: variables en columnas, observaciones en filas.

```
# Creando una "mini base de microdatos"
```

```
microdatos <- data.frame(
```

```
  id_persona = c(1, 2, 3),
```

```
  edad = c(34, 19, 52),
```

```
  ingreso = c(450000, 0, 780000),
```

```
  trabajo_semana_pasada = c(TRUE, FALSE, TRUE)
```

```
)
```

- Inspeccionamos el Data Frame.

```
str(microdatos)    # Estructura y tipos de datos de cada columna
```

```
summary(microdatos) # Resumen estadístico (min, media, max, etc.)
```

- Acceso a columnas específicas.

```
microdatos$ingreso # Extrae solo el vector de ingresos
```

Factores (Factors)

- Uso específico para datos categóricos de encuestas.

```
vector_estados <- c("Ocupado", "Desocupado", "Inactivo", "Ocupado")  
estado_factor <- factor(vector_estados)
```

- Visualización y manejo de los "Niveles".

```
levels(estado_factor) # Devuelve: "Desocupado" "Inactivo" "Ocupado"
```

```
# Forzar un orden lógico en los niveles (Ordinales)
```

```
nivel_edu <- factor(c("Secundario", "Universitario", "Primario"),  
  levels = c("Primario", "Secundario", "Universitario"),  
  ordered = TRUE)
```

Git

- Repo personal
 - git init (ya esta hecho cuando lo crearon)
 - git status
 - git add
 - git commit -m “actualiza tarea x / incluye nueva tarea y / ... ”
 - git push
- En carpeta de repo econ-520
 - git pull

Ejemplo tidyverse

Libro [R for Data Science \(2e\)](#)



- Capítulo 1 [Data visualization](#)
 - [Cheat sheets](#)



Tareas

- **Tarea 02 - R Base (programación)**

- Ejercicios de práctica en [R Tutorial](#) dados en clase (en diapo anterior)
 - Script R con códigos

- **Tarea 03 - Viz con tidyverse**

- Ejercicios del Capítulo 1 [Data visualization](#)
 - Script R con códigos